

Natural Language Processing — Introduction, Keras Intro, Basics of Deep Learning

Natural Language Processing

Sher Muhammad Daudpota

Sukkur IBA University

Course Introduction

- Course outlines
- Success stories
- Research Group (deep-nlp.net)

Success stories



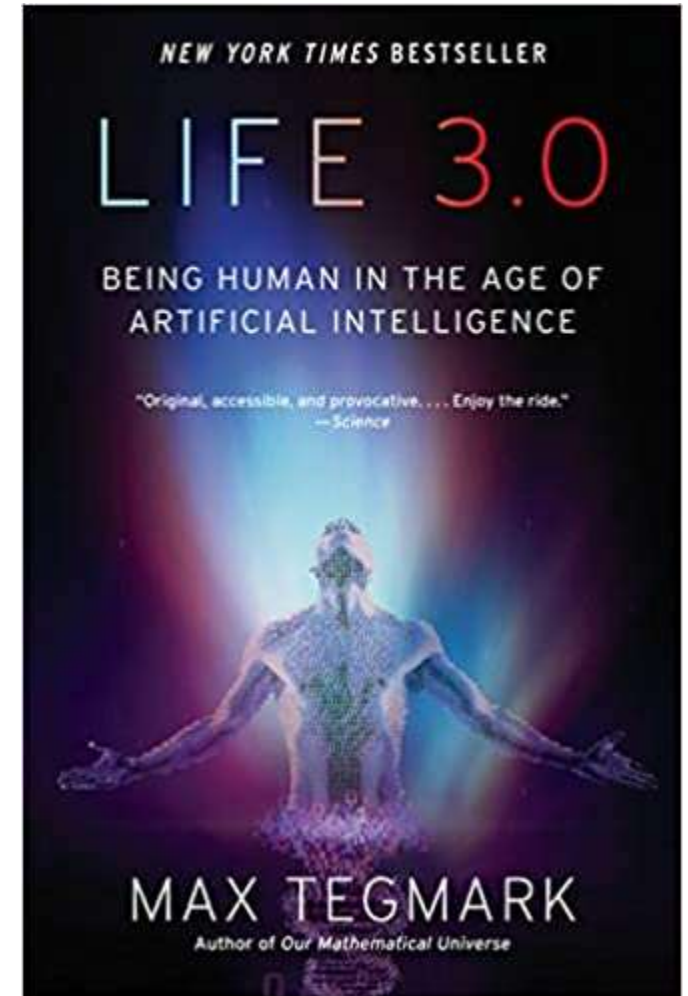
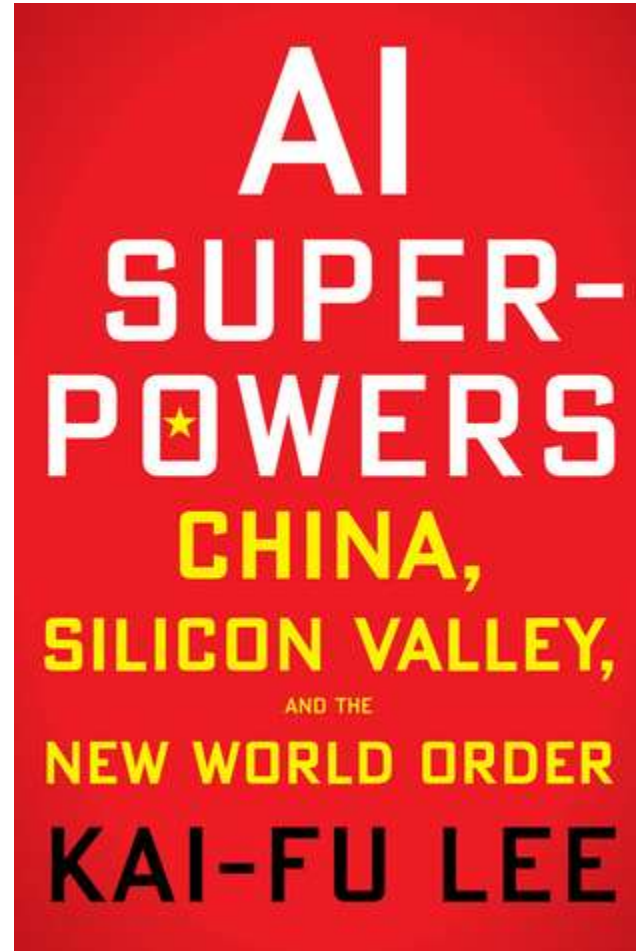
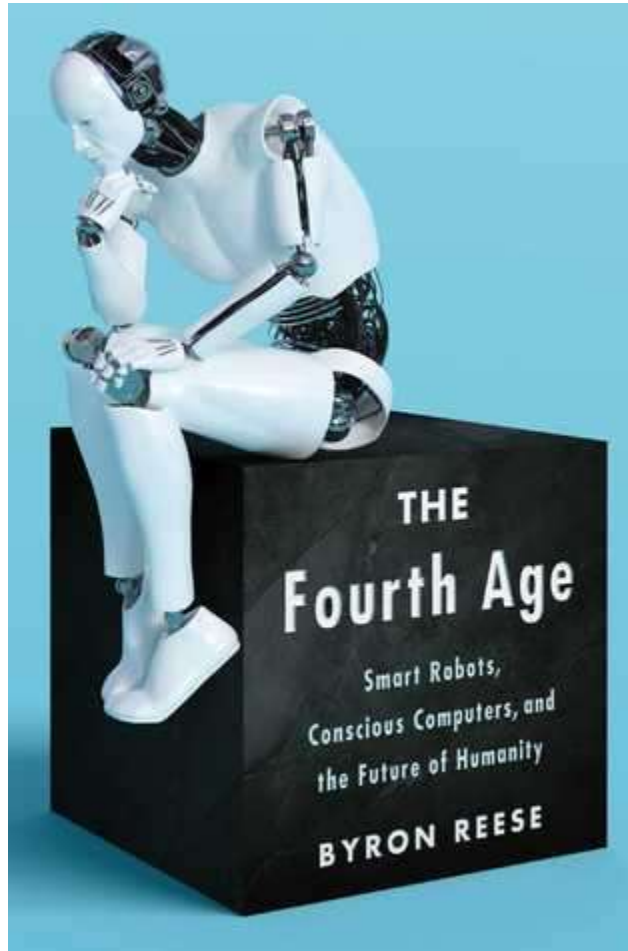
Success stories



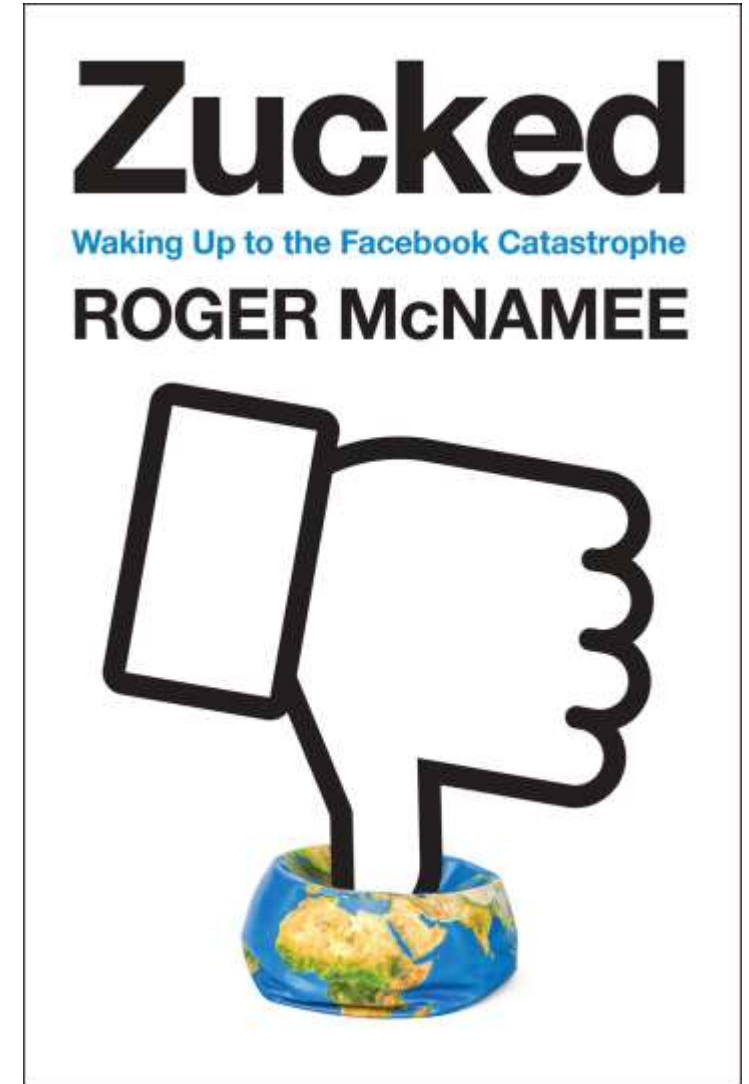
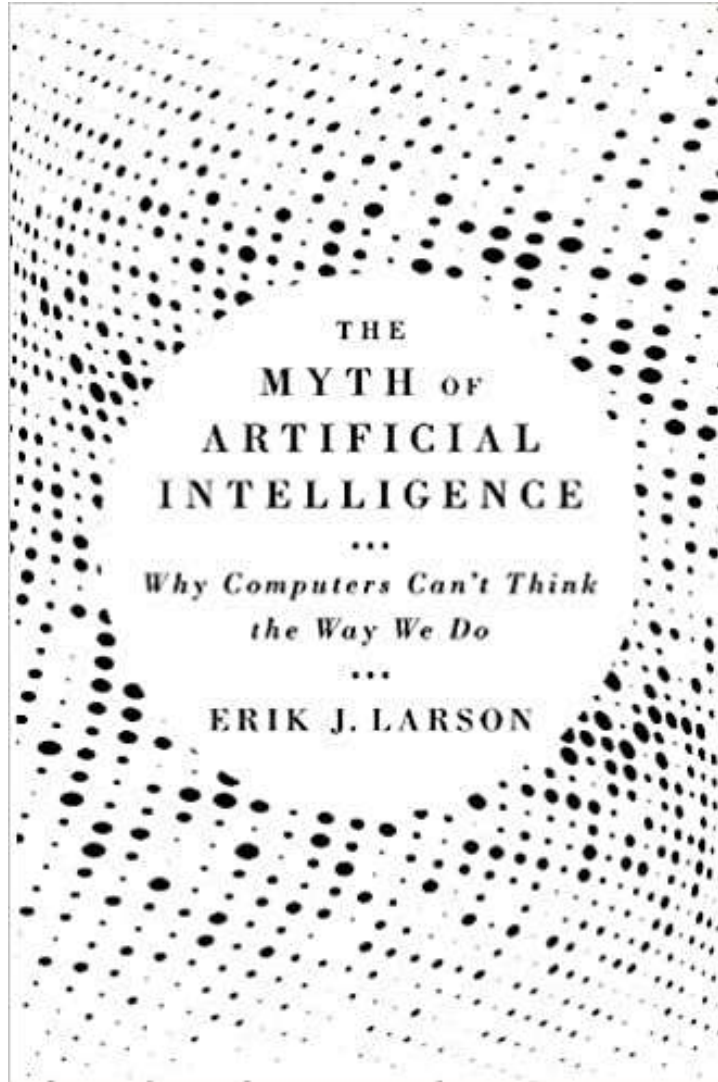
Success stories



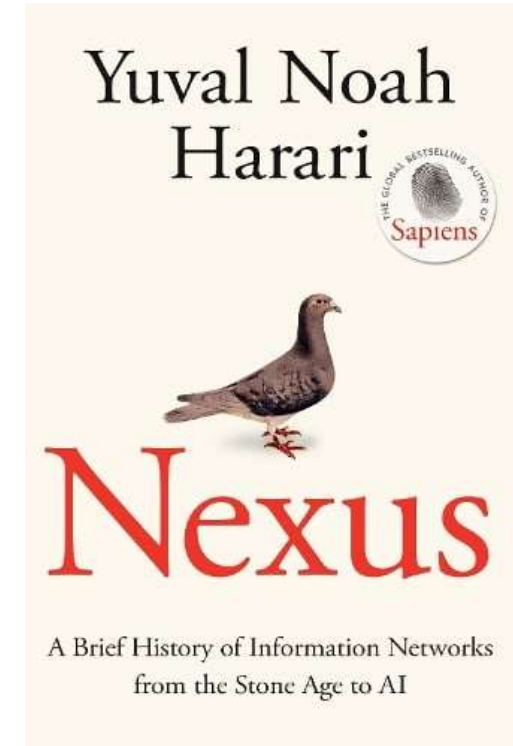
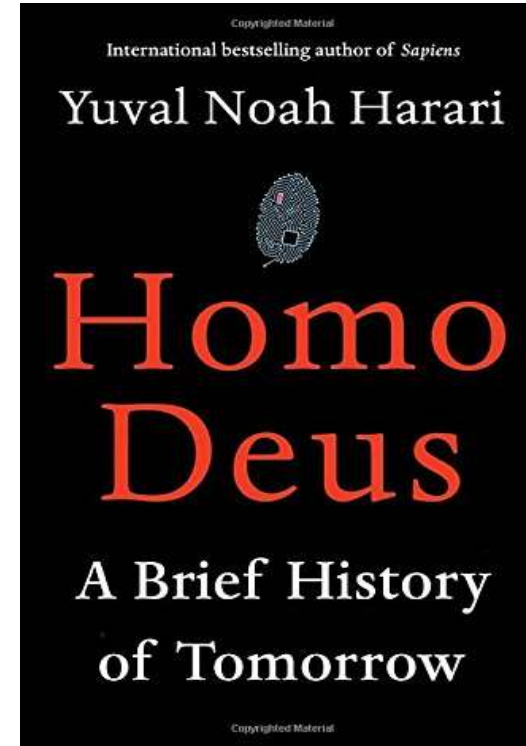
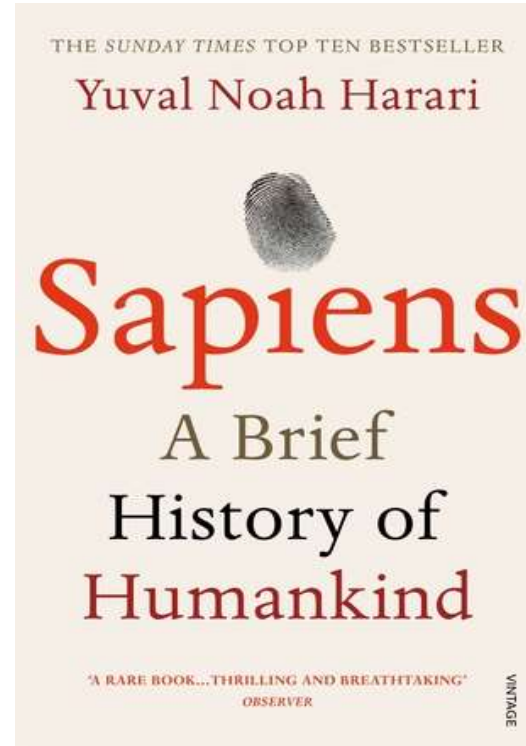
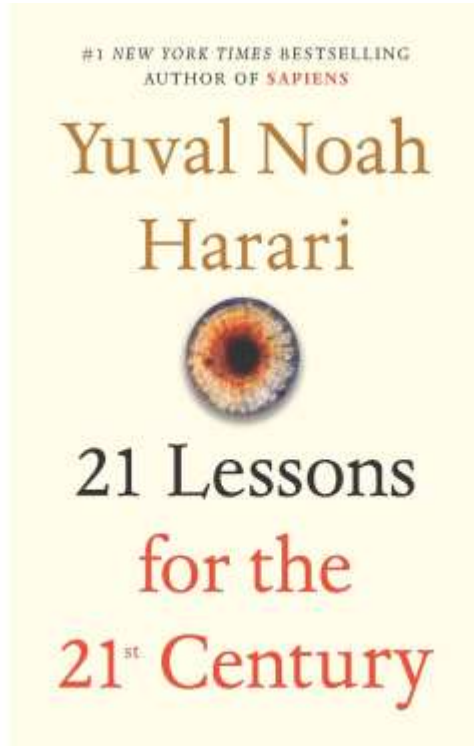
You must read....



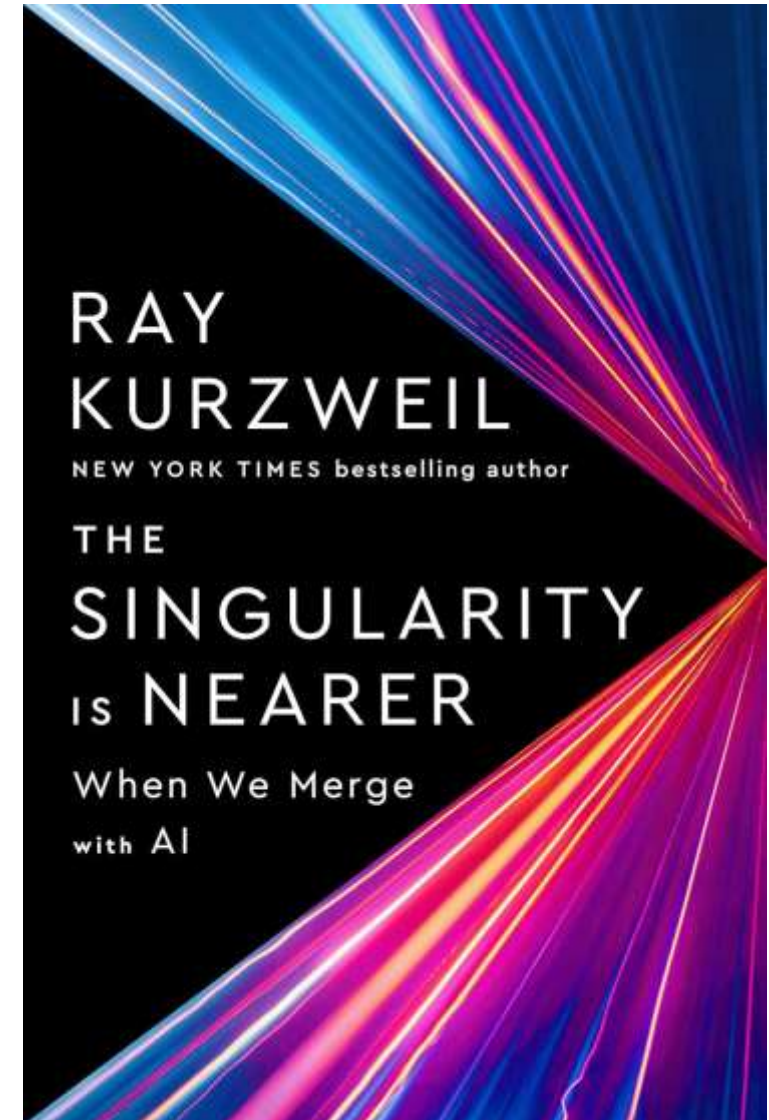
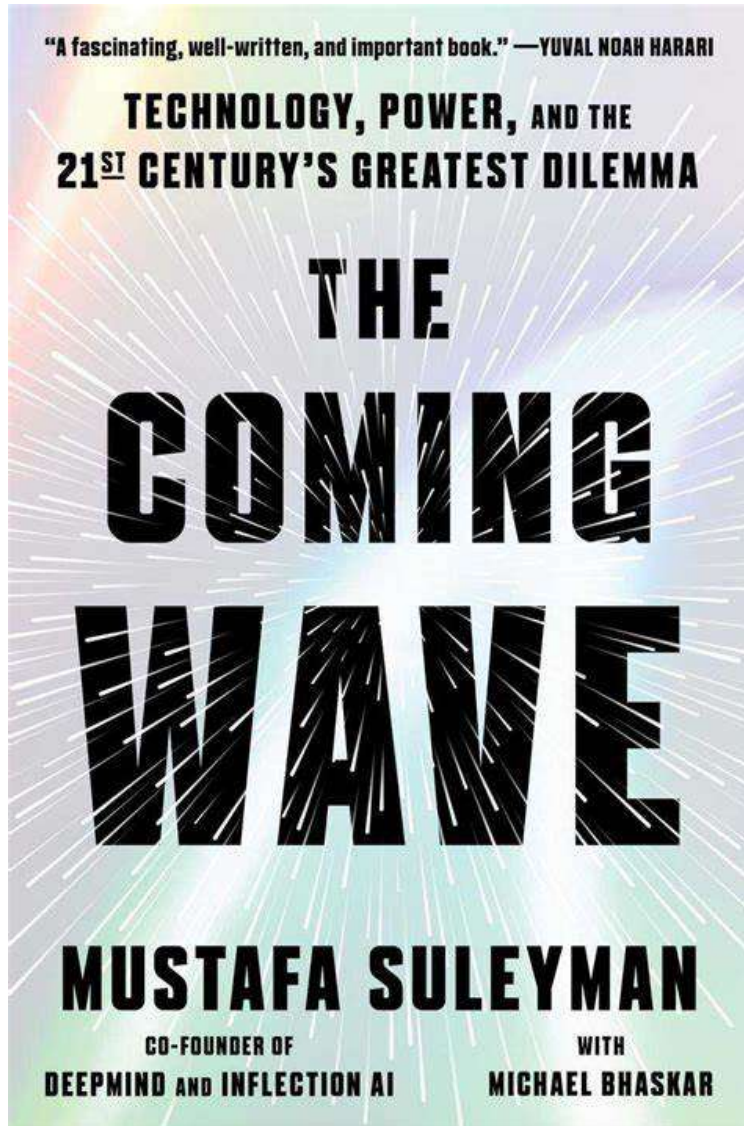
You must read....



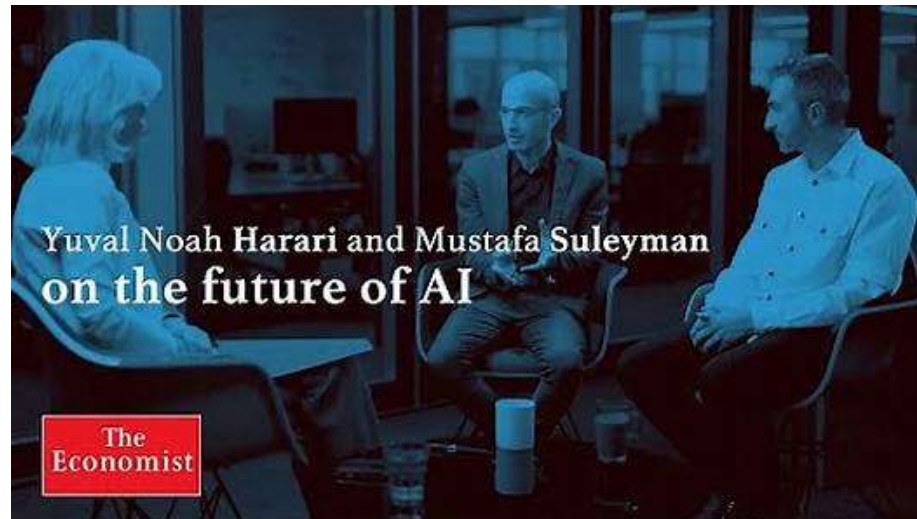
You must read....



You must read....



Must watch..



The gist of all these books..

- Human vs Artificial Super Intelligence
- Human extinction?

Turing Test

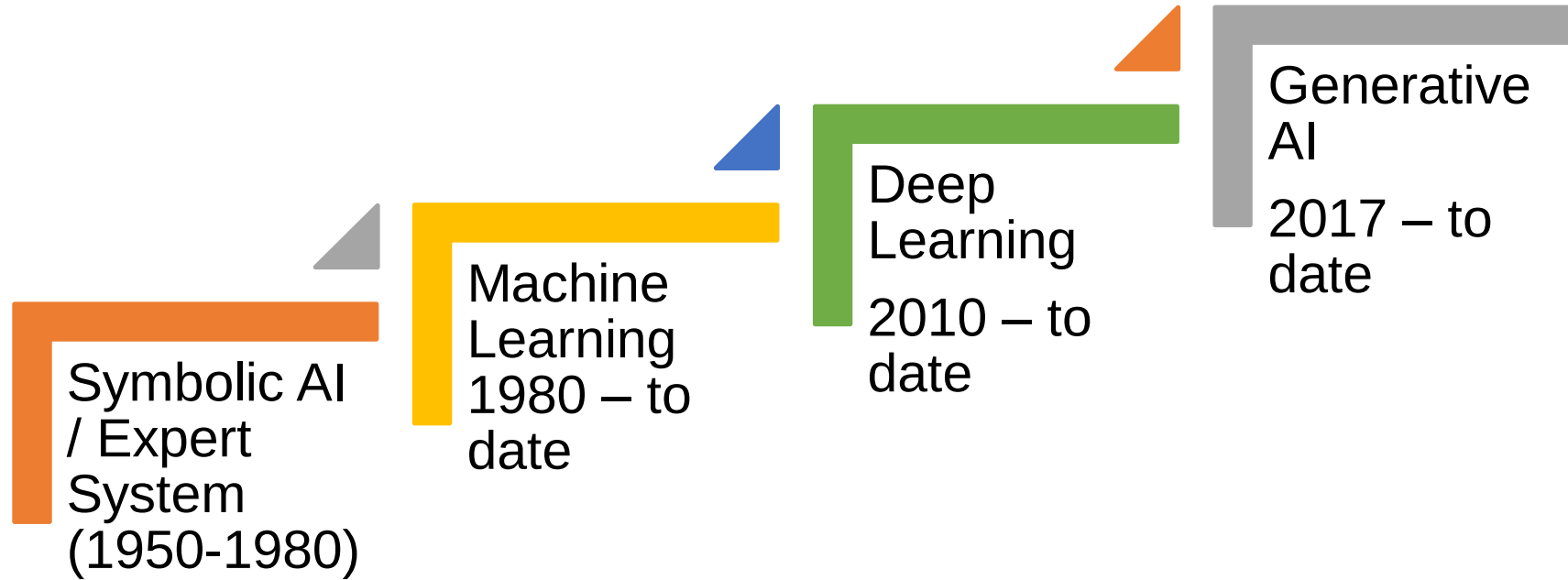
Proposed by **Alan Turing** in 1950, the test involves a human evaluator engaging in a natural language conversation with two unseen participants: one human and one machine. If the evaluator cannot reliably distinguish which participant is the machine based solely on their responses, the machine is said to have passed the test.

Revised Turing Test by Mustafa

Rather than focusing solely on whether a machine can mimic human responses in an open-ended conversation (as in the original Turing Test), Suleyman's approach suggests assessing AI systems based on their ability to solve specific, real-world problems and perform tasks effectively and ethically.

Example of earning \$1 million in a month's time

Journey of AI to Gen AI



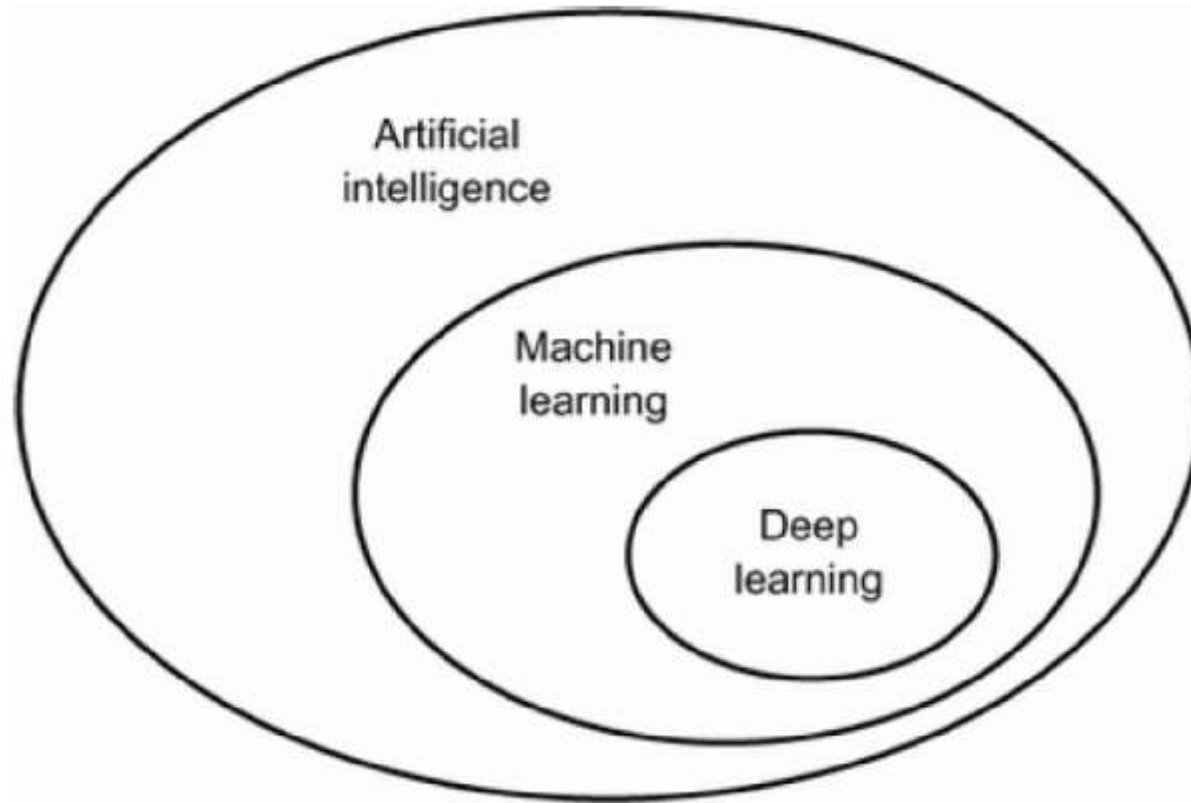
Deep Learning Introduction

- Outlines
 - Fundamental Concepts
 - Timeline of the development of machine learning
 - Why deep learning is getting popularity?
 - Keras Execution Environment
 - Hello World Example of Keras (MNIST Handwritten digit recognition/classification)

So, what are claims about AI?

- Intelligent chatbots
- Self-driving cars
- Virtual assistance
- IP address to each and every living thing
- Drama/TV scripts
- Music Composition
- Overall, they say jobs for human would be scarce !!!!

AI, Machine Learning and Deep Learning



Artificial Intelligence

- Born in 1950 when first the idea of “computer thinking” was coined

“the effort to automate intellectual task normally performed by human”

Artificial Intelligence

- *1956 Summer Workshop Proposal by John McCarthy, Assistant Professor of Mathematics Dartmouth College, USA*

The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer.

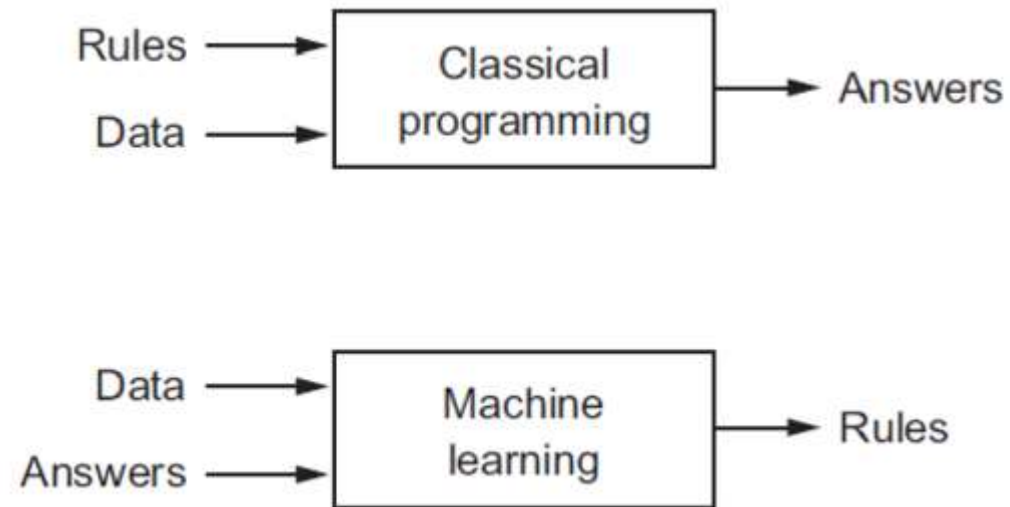
Symbolic AI and Expert System

- 1950-1980
- Alan Turing's "Turing Test"

*The Turing test, originally called the **imitation game** by Alan Turing in 1950, is a test of a machine's ability to exhibit intelligent behavior equivalent to, or indistinguishable from, that of a human.*

Machine Learning

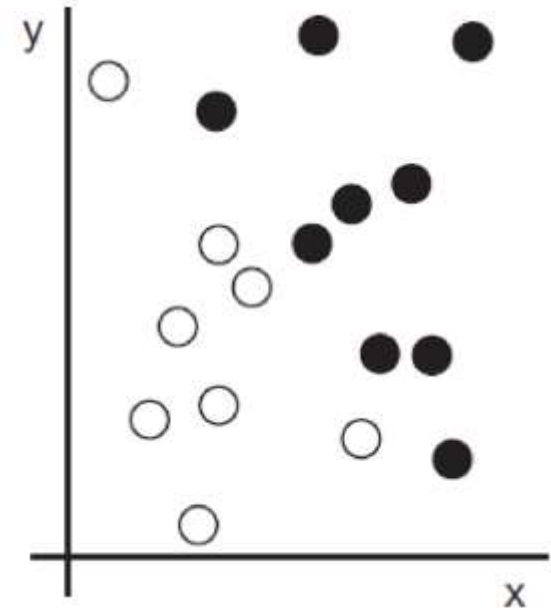
- Traditional Programming and Machine Learning



**Figure 1.2 Machine learning:
a new programming paradigm**

Machine Learning

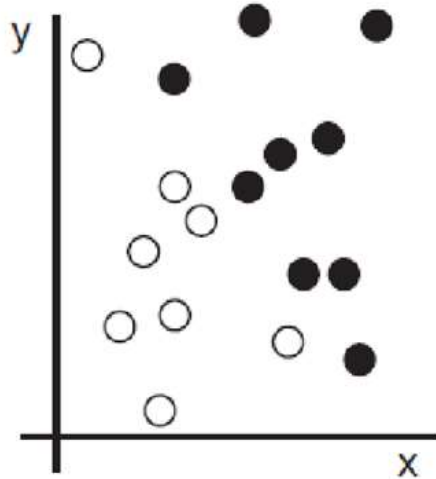
- Learning Representation from Data
- Machine Learning Requires,
 - Input data points
 - Examples of expected output
 - A way to measure performance



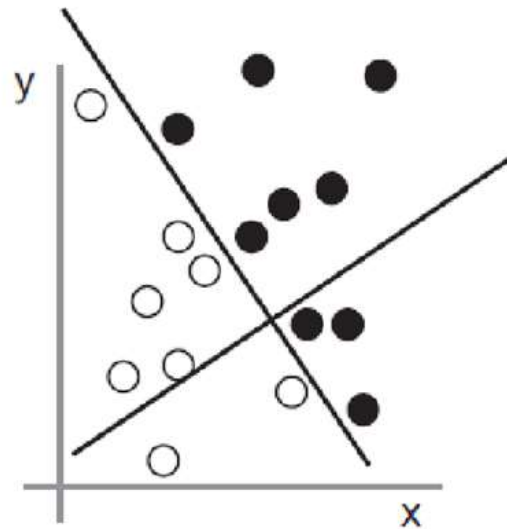
Machine Learning

- Learn representation

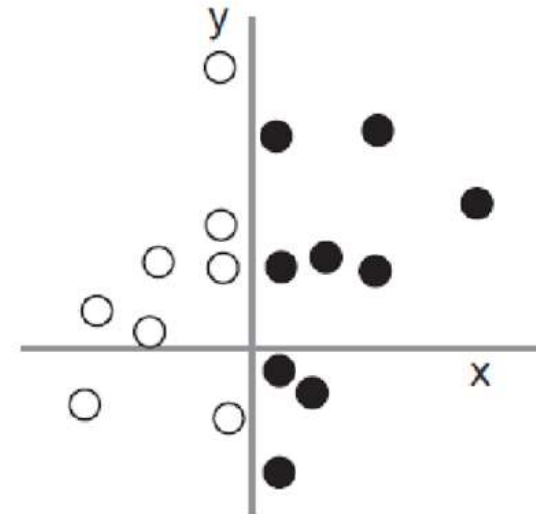
1: Raw data



2: Coordinate change



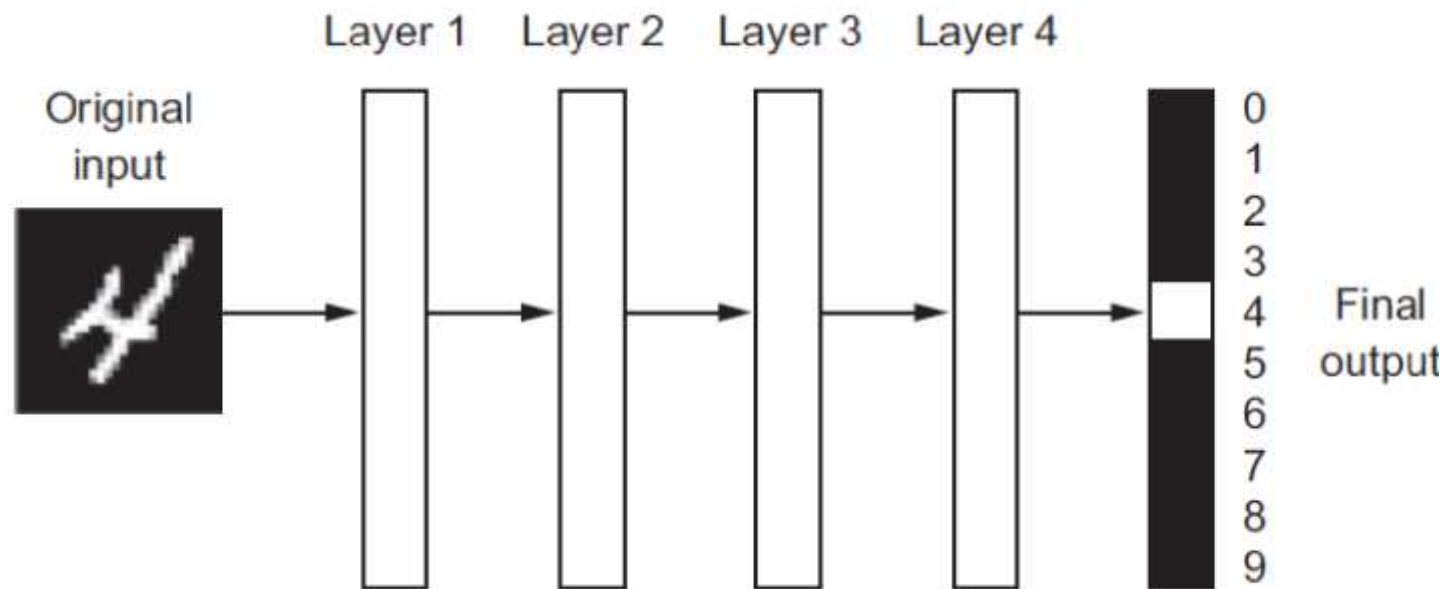
3: Better representation



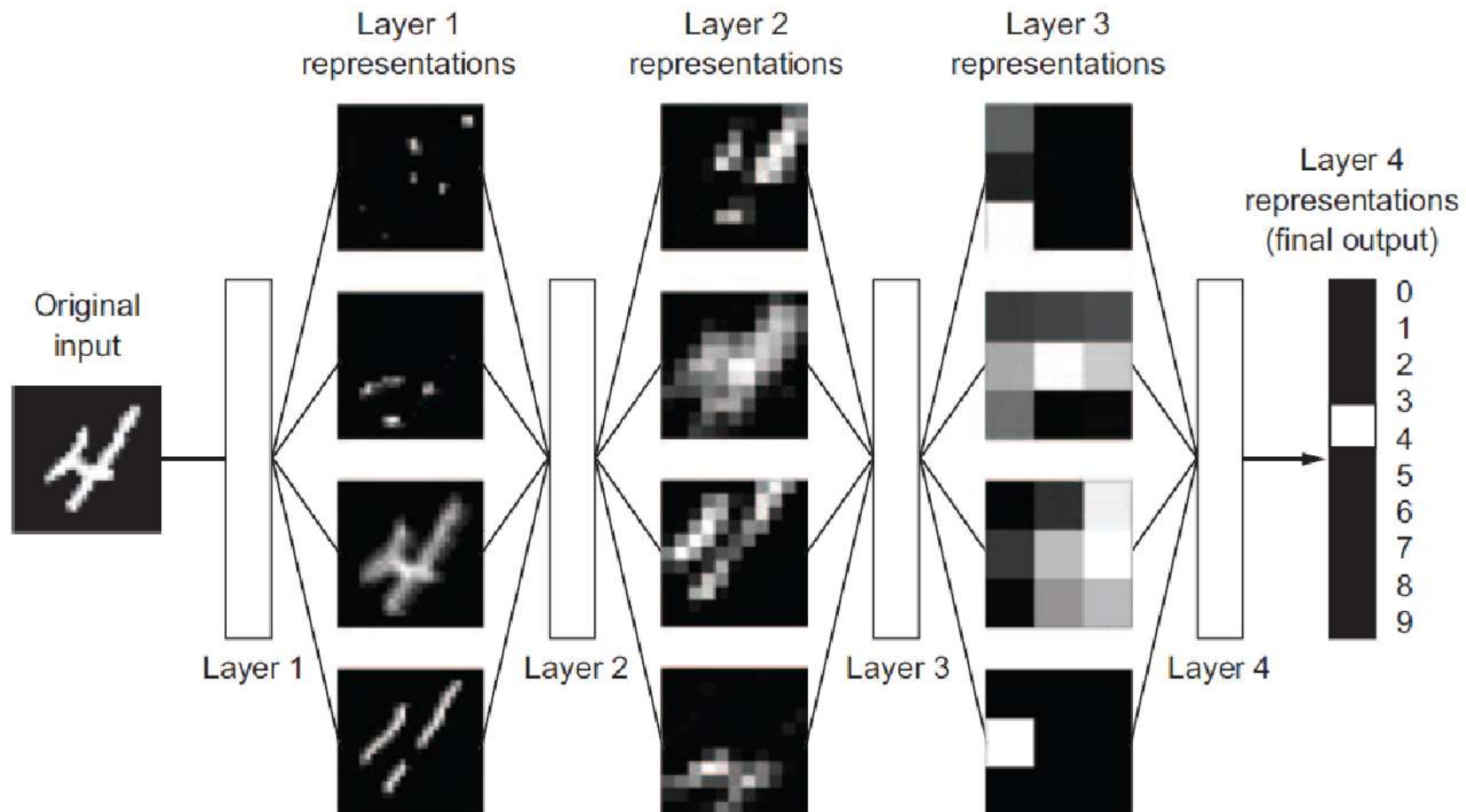
Deep Learning

- Deep Learning is subfield of Machine Learning
- Layered representation of learning
- Hierarchical representation of learning
- “Deep” in Deep Learning is number of layers or depth of hierarchy
- Layered representation is learned through *neural network*
- *Remember people will say neural networks are modeled the way human brain works, but believe me that is not the truth...*

Deep Learning



Deep Learning



Deep Learning

- Three important things in deep learning,
 - Weights
 - Loss Function
 - Optimizer

Deep Learning: Weights

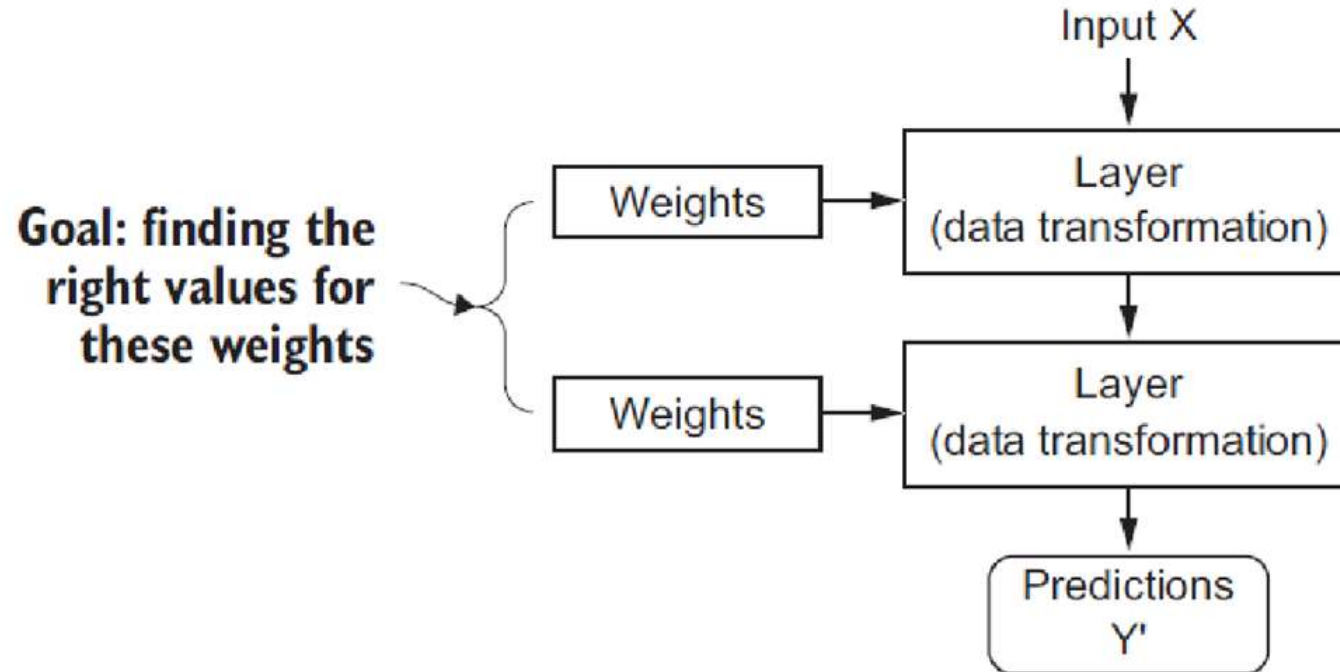


Figure 1.7 A neural network is parameterized by its weights.

Deep Learning: Loss Function

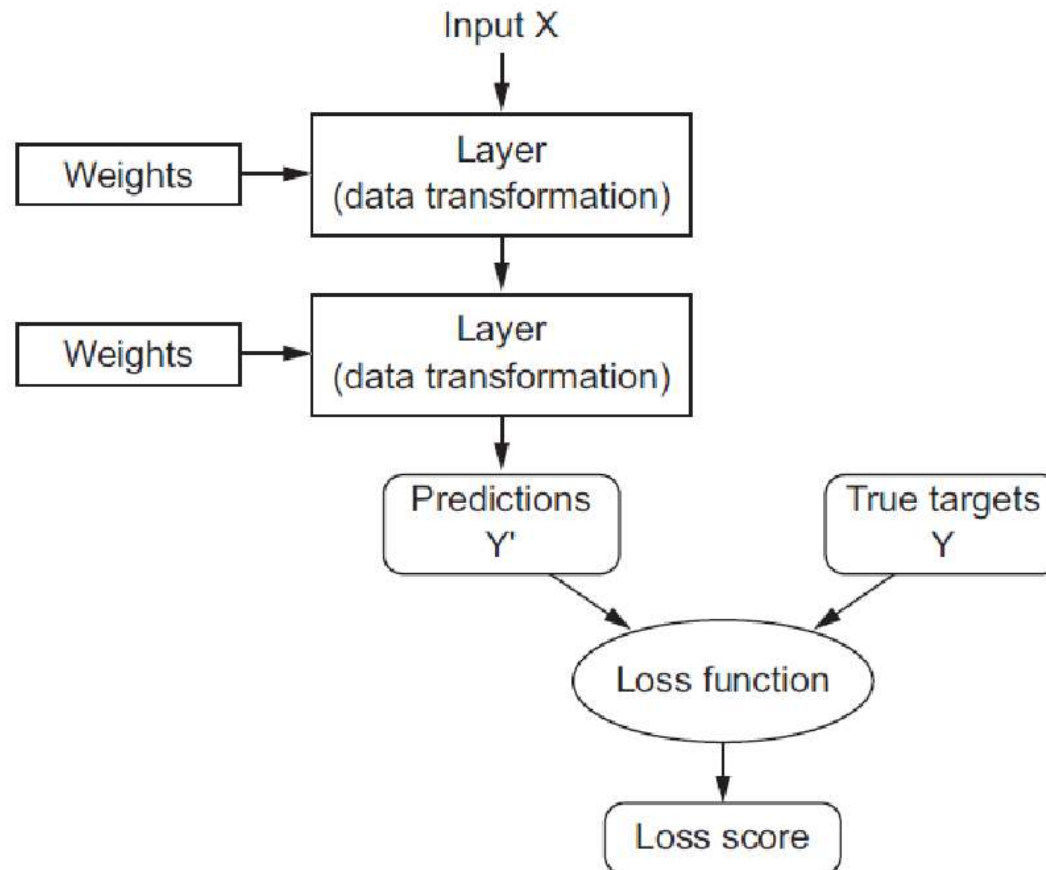


Figure 1.8 A loss function measures the quality of the network's output.

Deep Learning: Optimizer

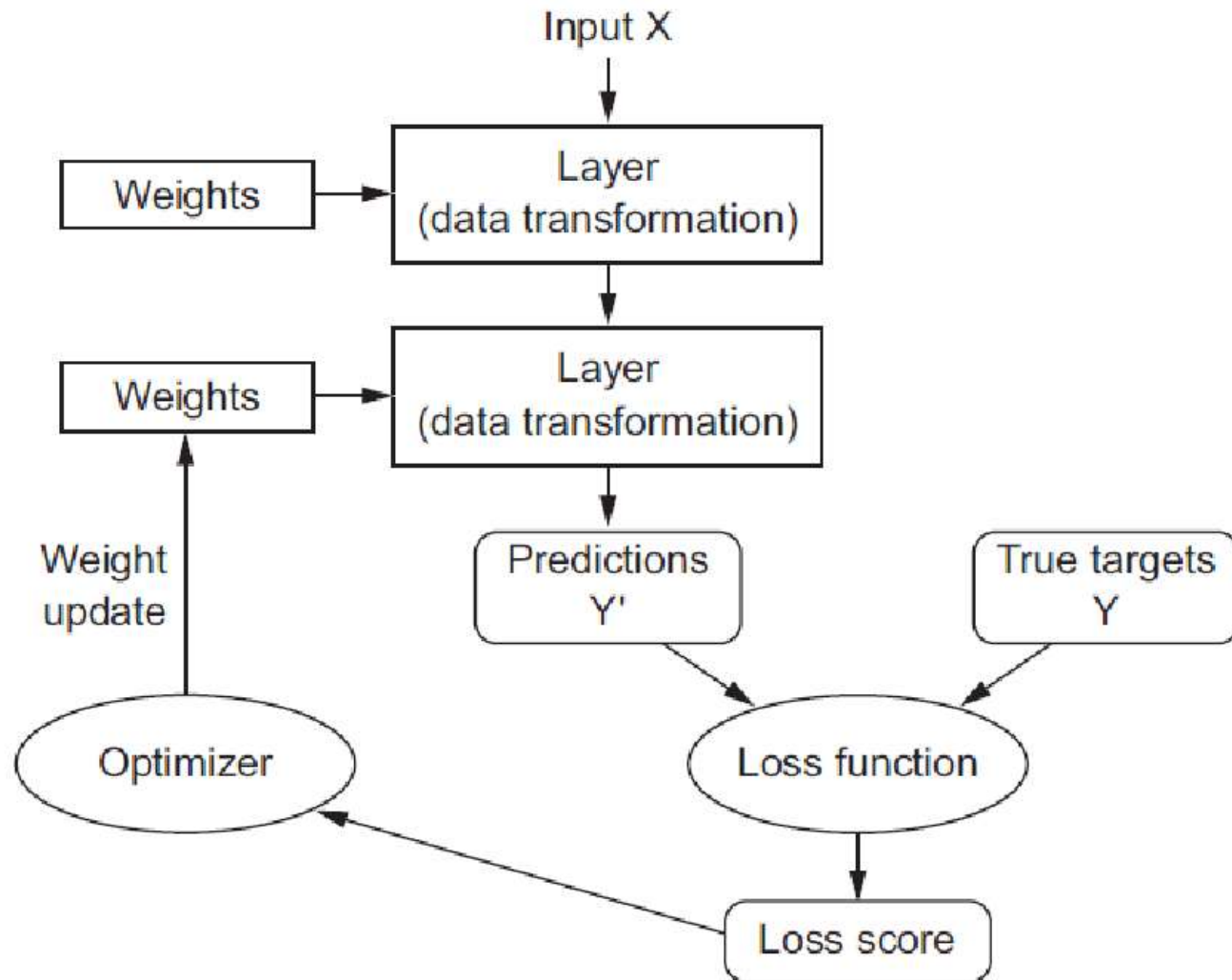


Figure 1.9 The loss score is used as a feedback signal to adjust the weights.

Deep Learning Wonders So far..

- Near-human-level image classification
- Near-human-level speech recognition
- Near-human-level handwriting transcription
- Improved machine translation

Deep Learning Wonders So far..

- Improved text-to-speech conversion
- Digital assistants such as Google Now and Amazon Alexa
- Near-human-level autonomous driving
- Improved ad targeting, as used by Google, Baidu, and Bing
- Improved search results on the web
- Ability to answer natural-language questions
- Superhuman Go playing

Don't over hype....

- Else funding will dry up.. AI Winter..
- 1960 Symbolic AI
 - 1970 claims by Marvin Minsky
- 1980 Expert System
 - Over \$1 billion each year on technology
 - By 90s, companies realized its waste of money...
- 2010.... No more AI winters, hopefully..

The AI Promises

- AI is still in supplemental stage..
 - Wishing you birthday
 - Showing you route
 - Some assistance in medical
 - Loads of other aspects where AI is supplemental
- A transition is yet to take place to come at center of our lives, the way we think, work and live..

History of Machine Learning

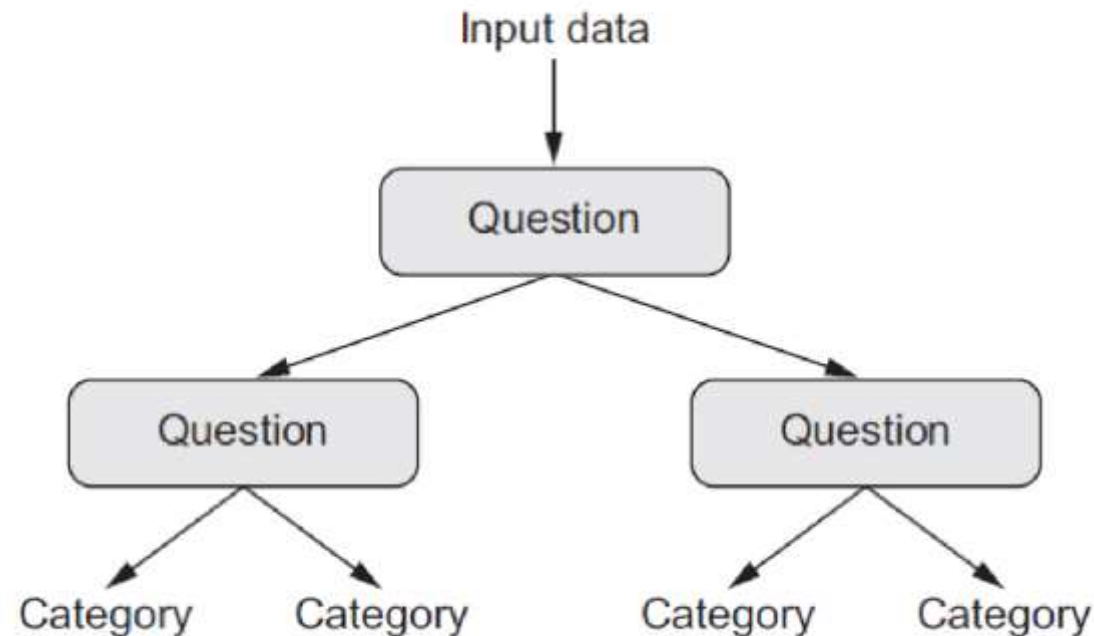
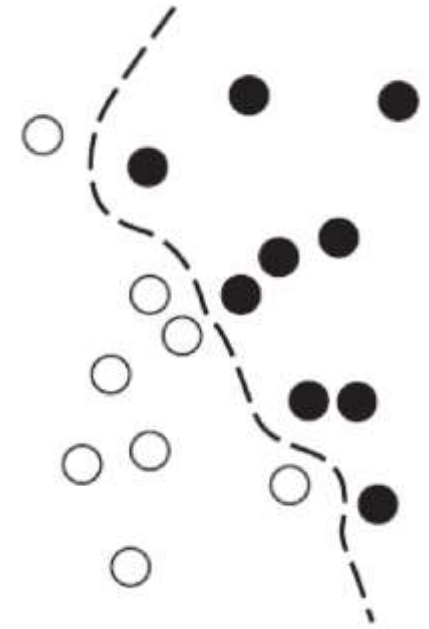
- Not every machine learning problem is deep learning problem
- Don't think deep learning a hammer and every ML problem a nail..
- Must must not fall in this trap to approach every machine learning problem from DL aspect..
- The only way to avoid trap is to learn other traditional approaches
- Remember deep learning mean requirement of loads of data for training...

History of Machine Learning

- Probabilistic models
 - Naïve Bayes Algorithm
 - Logistic Regression
- Early Neural Networks
 - Initially coined in 1950s
 - Problem with training large network that was solved in 1980s through back propagation
 - 1989 first major NN success through LeNet at Bell labs for handwritten digit recognition

History of Machine Learning

- Kernel methods
 - Support Vector Machine (decision boundary)
- Decision trees, random forest and gradient boosting machines
 - Random Forest
 - Gradient Boosting



History of Machine Learning

- Back to Neural Network
 - 2012 watershed moment for Neural Network: ImageNet
 - ImageNet (1.4 million images with 1000 classes)
 - 2010 Accuracy of traditional computer vision approaches was 74.3%
 - Deep Neural Network gave 96.4%
 - Convnets are now goto solution for computer vision problems

What makes deep learning different or so successful?

- Feature Engineering
- DL automates it

Modern ML landscape

- Must be using Kaggle
- By 2016, 2017, two most popular algorithm on Kaggle were
 - Gradient boosting machines
 - Deep learning
- Now GBM has taken back seat, DL is all what Kaggle sees most of the times for many tasks of,
 - Computer Vision
 - NLP
 - Audio Processing
 - And almost everything 😊

Why Deep Learning Now?

- Hardware
 - GPU, NVIDIA's CUDA and TITAN X
 - Google's TPU (Tensor Pro.....)
- Datasets and benchmarks
- Algorithmic advancements
 - Better activation functions
 - Weight initialization schemes
 - Optimization schemes

Results? Investment

Machine learning—in particular, deep learning—has become central to the product strategy of these tech giants. In late 2015, Google CEO Sundar Pichai stated, “Machine learning is a core, transformative way by which we’re rethinking how we’re doing everything. We’re thoughtfully applying it across all our products, be it search, ads, YouTube, or Play. And we’re in early days, but you’ll see us—in a systematic way—apply machine learning in all these areas.”⁷

Let's run MNIST hand written digits recognition..

Hello World of Deep Learning 😊



Data Representation for Tensors

- Tensor is a container for data, mostly numerical data
- Matrices are 2D tensors
- Scalars
 - Tensor with only one number
 - 0D tensor

```
>>> import numpy as np
>>> x = np.array(12)
>>> x
array(12)
>>> x.ndim
0
```

Data Representation for Tensors

- Vectors
 - 1D tensors

```
>>> x = np.array([12, 3, 6, 14])  
>>> x  
array([12, 3, 6, 14])  
>>> x.ndim  
1
```

Data Representation for Tensors

- Matrices (2D tensors)
 - Array of vectors

```
>>> x = np.array([[5, 78, 2, 34, 0],  
                  [6, 79, 3, 35, 1],  
                  [7, 80, 4, 36, 2]])
```

```
>>> x.ndim  
2
```

Data Representation for Tensors

- 3D tensor

```
>>> x = np.array([[[5, 78, 2, 34, 0],  
                  [6, 79, 3, 35, 1],  
                  [7, 80, 4, 36, 2]],  
                 [[5, 78, 2, 34, 0],  
                  [6, 79, 3, 35, 1],  
                  [7, 80, 4, 36, 2]],  
                 [[5, 78, 2, 34, 0],  
                  [6, 79, 3, 35, 1],  
                  [7, 80, 4, 36, 2]])
```

```
>>> x.ndim
```

```
3
```

Data Representation for Tensors

- Tensor Key Attributes
 - Number of axes: that is returned through *ndim*
 - Shape: define dim along each axis and returned through *shape*
 - Data type: It can be *float32*, *uint8*, *int32*, *float64* and *char*

Data Representation for Tensors

- Real-world tensors examples

- *Vector data*—2D tensors of shape (samples, features)
- *Timeseries data or sequence data*—3D tensors of shape (samples, timesteps, features)
- *Images*—4D tensors of shape (samples, height, width, channels) or (samples, channels, height, width)
- *Video*—5D tensors of shape (samples, frames, height, width, channels) or (samples, frames, channels, height, width)

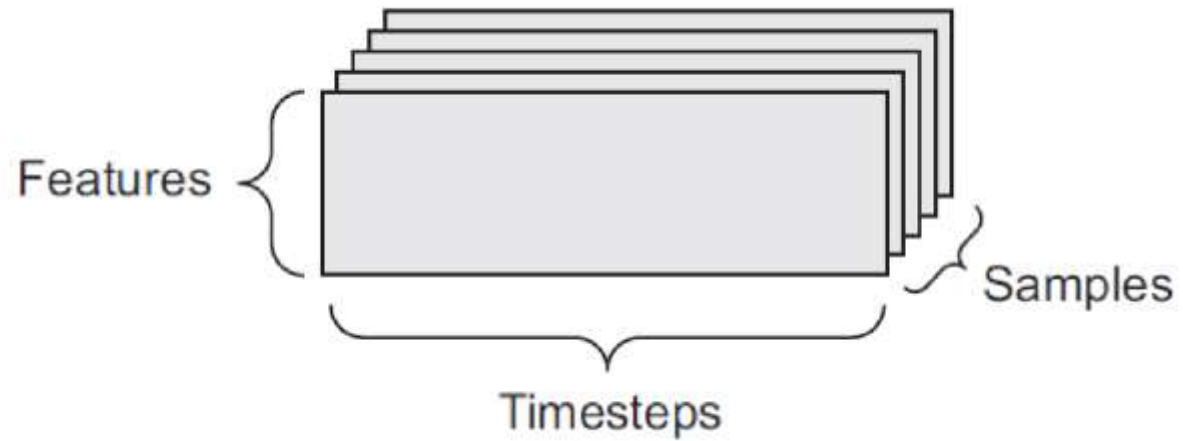
Data Representation for Tensors

- Vector Data

- An actuarial dataset of people, where we consider each person's age, ZIP code, and income. Each person can be characterized as a vector of 3 values, and thus an entire dataset of 100,000 people can be stored in a 2D tensor of shape $(100000, 3)$.
- A dataset of text documents, where we represent each document by the counts of how many times each word appears in it (out of a dictionary of 20,000 common words). Each document can be encoded as a vector of 20,000 values (one count per word in the dictionary), and thus an entire dataset of 500 documents can be stored in a tensor of shape $(500, 20000)$.

Data Representation for Tensors

- Timeseries data (3D tensor)

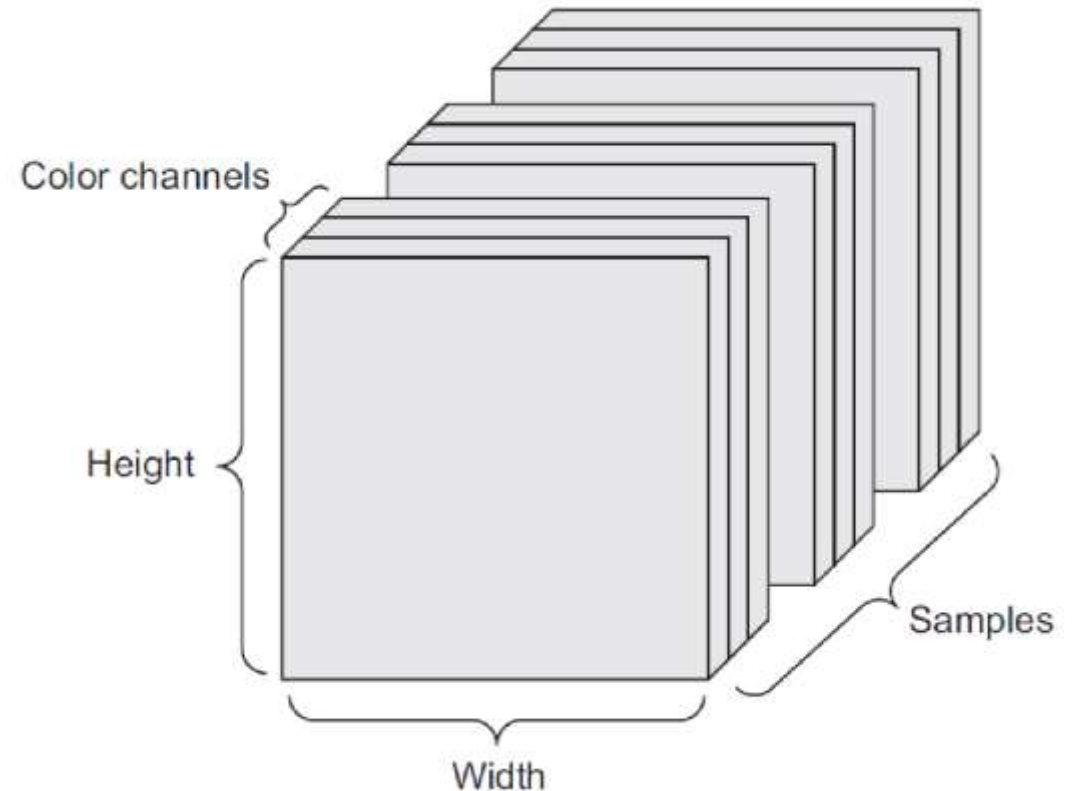


Data Representation for Tensors

- A dataset of stock prices. Every minute, we store the current price of the stock, the highest price in the past minute, and the lowest price in the past minute. Thus every minute is encoded as a 3D vector, an entire day of trading is encoded as a 2D tensor of shape $(390, 3)$ (there are 390 minutes in a trading day), and 250 days' worth of data can be stored in a 3D tensor of shape $(250, 390, 3)$. Here, each sample would be one day's worth of data.
- A dataset of tweets, where we encode each tweet as a sequence of 280 characters out of an alphabet of 128 unique characters. In this setting, each character can be encoded as a binary vector of size 128 (an all-zeros vector except for a 1 entry at the index corresponding to the character). Then each tweet can be encoded as a 2D tensor of shape $(280, 128)$, and a dataset of 1 million tweets can be stored in a tensor of shape $(1000000, 280, 128)$.

Data Representation for Tensors

- Image Data (4D tensors)
 - A batch of 128 grayscale images of size 256 x 256 can be stored in a tensor of shape (128,256,256,1)
 - A batch of 128 color images could be stored in a tensor of shape (128,256,256,3)



Data Representation for Tensors

- Video Data (5D tensors)

`(samples, frames, height, width, color_depth)`

For instance, a 60-second, 144×256 YouTube video clip sampled at 4 frames per second would have 240 frames. A batch of four such video clips would be stored in a tensor of shape `(4, 240, 144, 256, 3)`. That's a total of 106,168,320 values! If the dtype of the tensor was `float32`, then each value would be stored in 32 bits, so the tensor would represent 405 MB. Heavy! Videos you encounter in real life are much lighter, because they aren't stored in `float32`, and they're typically compressed by a large factor (such as in the MPEG format).

Data Representation for Tensors

- Example

```
from keras.datasets import mnist
```

```
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

- ndim

```
>>> print(train_images.ndim)  
3
```

- Shape and dtype

```
>>> print(train_images.shape)  
(60000, 28, 28)
```

```
>>> print(train_images.dtype)  
uint8
```

Data Representation for Tensors

```
digit = train_images[4]

import matplotlib.pyplot as plt
plt.imshow(digit, cmap=plt.cm.binary)
plt.show()
```

Data Representation for Tensors

- Tensor slicing is selecting specific elements in an tensor,

```
>>> my_slice = train_images[10:100]
>>> print(my_slice.shape)
(90, 28, 28)
```

```
>>> my_slice = train_images[10:100, :, :] ← Equivalent to the
                                             previous example
```

```
>>> my_slice.shape
(90, 28, 28)
```

```
>>> my_slice = train_images[10:100, 0:28, 0:28] ← Also equivalent to the
                                                    previous example
```

```
>>> my_slice.shape
(90, 28, 28)
```

```
my_slice = train_images[:, 14:, 14:]
```


Data Representation for Tensors

- Data batches

```
batch = train_images[:128]
```

```
batch = train_images[128:256]
```

MNIST Complete Example

Classifying movie reviews (IMDB Practical)

Classifying newswire: A multiclass classification (practical)

Playing with BBC dataset

Assignment

- Available on eLearning
 - Tweet Emotion Classification
 - BBC Sports News Classification (try doing it in class, if time permits)

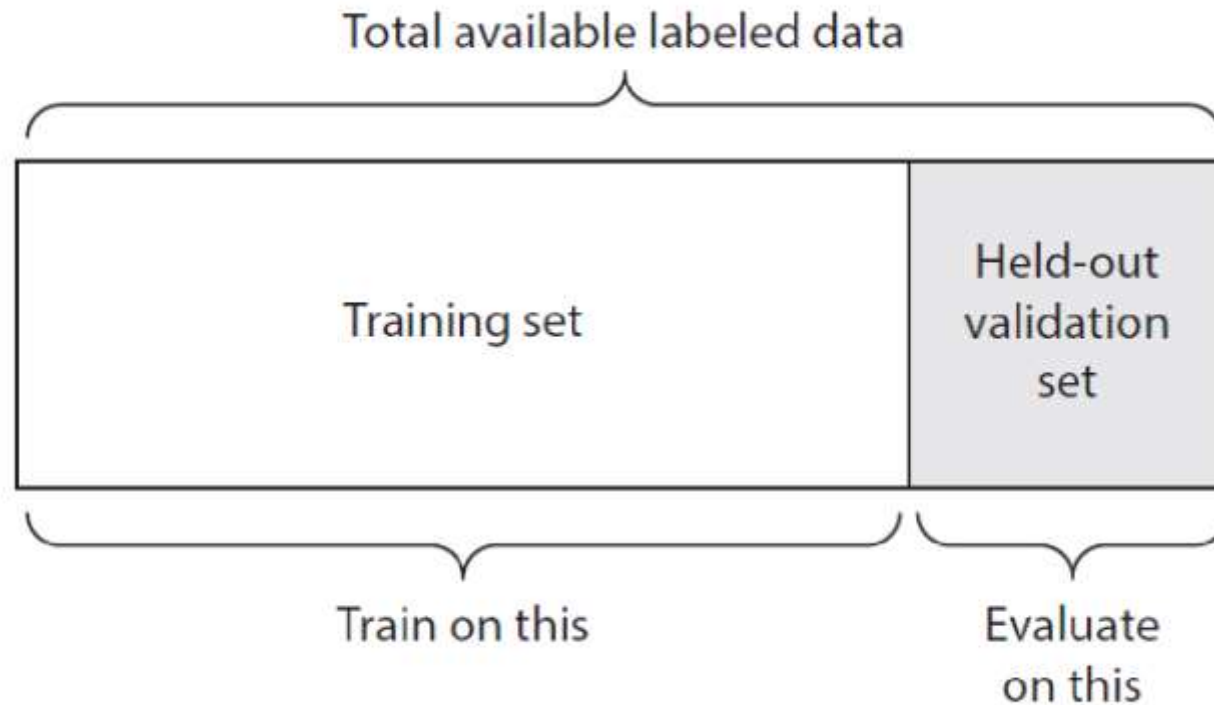
Fundamentals of Machine Learning

- Four Branches of Machine Learning
 - Supervised Learning
 - Sequence Generation
 - Syntax Tree Prediction
 - Object Detection
 - Image Segmentation
 - Unsupervised Learning
 - Dimensionality Reduction
 - Clustering
 - Self Supervised Learning
 - Reinforcement Learning

Evaluating Machine Learning Models

- Training set
- Validation Set
- Test Set

Simple Hold Out Validation



Simple Hold Out Validation

```
num_validation_samples = 10000
```

```
np.random.shuffle(data)
```

Shuffling the data is
usually appropriate.

```
validation_data = data[:num_validation_samples]
```

Defines the
validation set

```
data = data[num_validation_samples:]
```

Defines the training set

```
training_data = data[:]
```

```
model = get_model()
```

```
model.train(training_data)
```

```
validation_score = model.evaluate(validation_data)
```

Trains a model on the training
data, and evaluates it on the
validation data

```
# At this point you can tune your model,
```

```
# retrain it, evaluate it, tune it again...
```

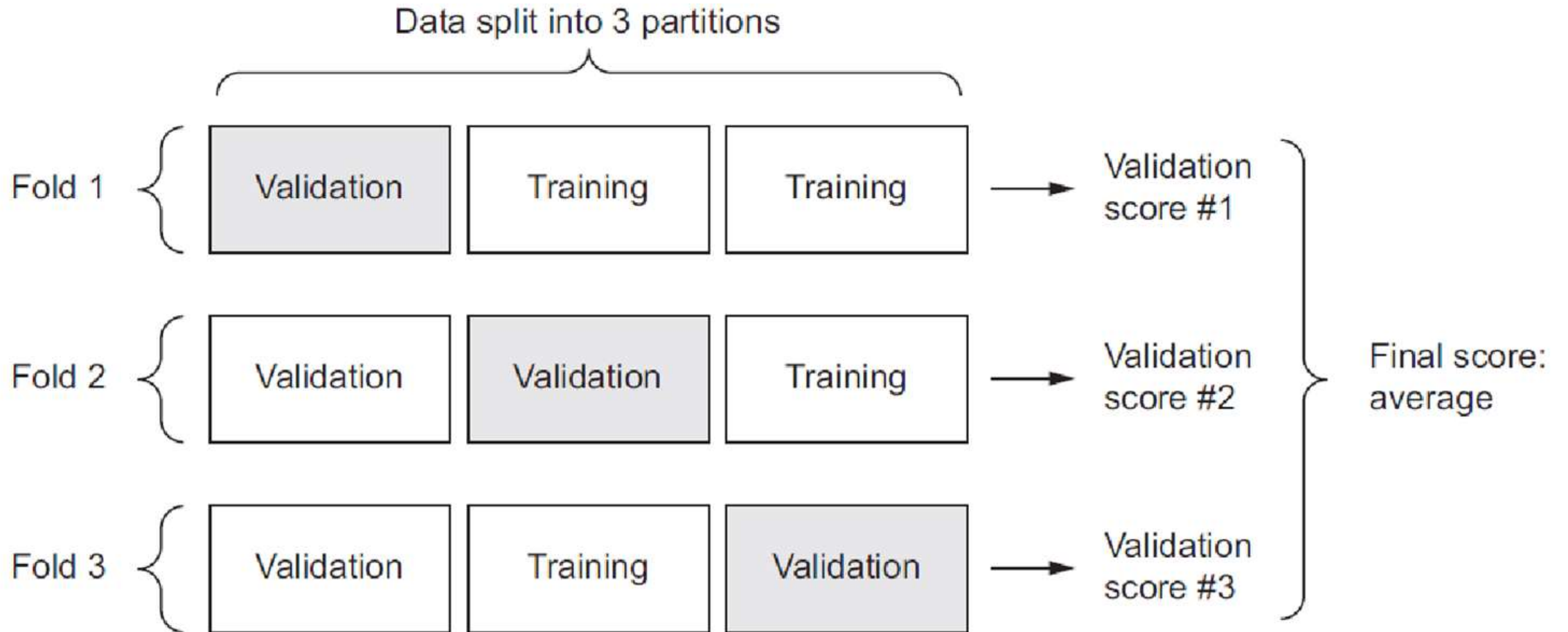
```
model = get_model()
```

```
model.train(np.concatenate([training_data,  
                             validation_data]))
```

```
test_score = model.evaluate(test_data)
```

Once you've tuned your
hyperparameters, it's common to
train your final model from scratch
on all non-test data available.

K-Fold Validation



```
k = 4
num_validation_samples = len(data) // k
```

```
np.random.shuffle(data)
```

```
validation_scores = []
```

```
for fold in range(k):
```

```
    validation_data = data[num_validation_samples * fold:
        num_validation_samples * (fold + 1)]
```

```
    training_data = data[:num_validation_samples * fold] +
        data[num_validation_samples * (fold + 1):]
```

```
    model = get_model()
```

```
    model.train(training_data)
```

```
    validation_score = model.evaluate(validation_data)
```

```
    validation_scores.append(validation_score)
```

**Selects the validation-
data partition**

**Uses the remainder of the data
as training data. Note that the
+ operator is list concatenation,
not summation.**

**Creates a brand-new instance
of the model (untrained)**

```
validation_score = np.average(validation_scores)
```

```
model = get_model()
```

```
model.train(data)
```

```
test_score = model.evaluate(test_data)
```

**Trains the final
model on all non-
test data available**

**Validation score:
average of the
validation scores
of the k folds**

Keep in mind....

- Data Representation in Cross Validation
- Arrow of time
- Redundancy in your data

Data Preprocessing, feature engineering and feature learning

- Preprocessing
 - Vectorization
 - Value normalization
 - Handling missing values
 - Feature Engineering

Overfitting, Underfitting, Optimization, Generalization

- Regularization
 - Process to fight overfitting
- Too much capacity vs memory starvation

Overfitting, Underfitting, Optimization, Generalization

Listing 4.3 Original model

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

Now let's try to replace it with this smaller network.

Listing 4.4 Version of the model with lower capacity

```
model = models.Sequential()
model.add(layers.Dense(4, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(4, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```


Overfitting, Underfitting, Optimization, Generalization

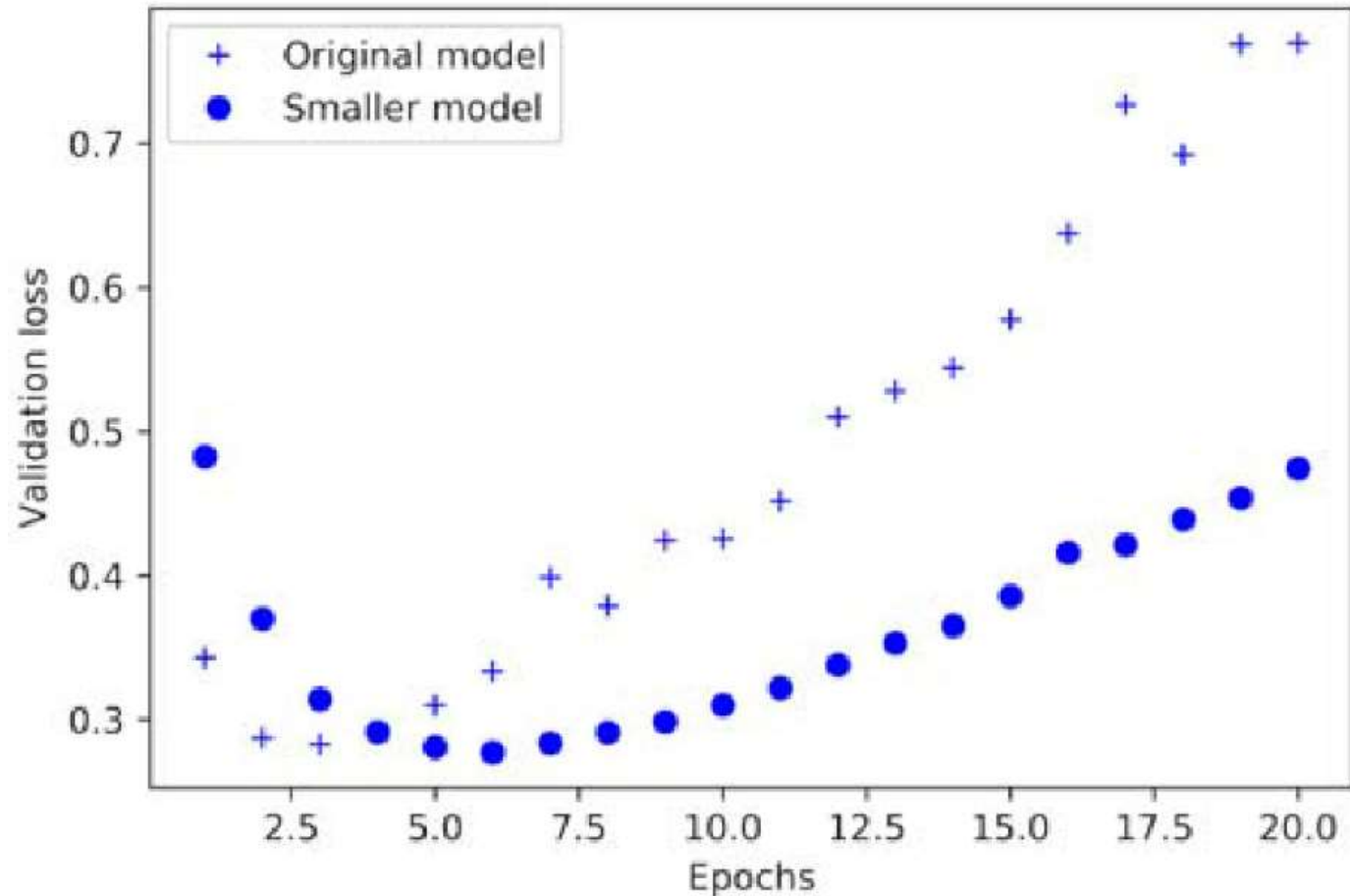


Figure 4.4 Effect of model capacity on validation loss: trying a smaller model

Overfitting, Underfitting, Optimization, Generalization

Listing 4.5 Version of the model with higher capacity

```
model = models.Sequential()  
model.add(layers.Dense(512, activation='relu', input_shape=(10000,)))  
model.add(layers.Dense(512, activation='relu'))  
model.add(layers.Dense(1, activation='sigmoid'))
```

Overfitting, Underfitting, Optimization, Generalization

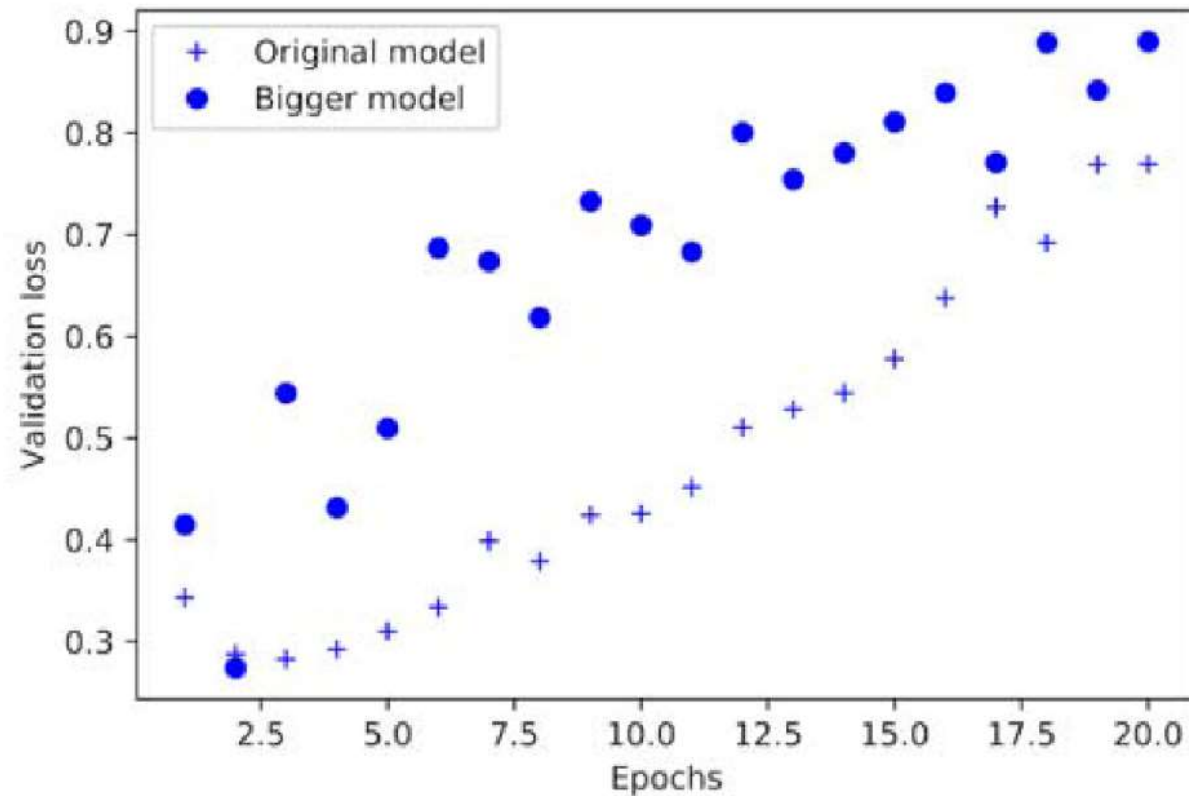


Figure 4.5 Effect of model capacity on validation loss: trying a bigger model

Regularization

- *L1 regularization*—The cost added is proportional to the *absolute value of the weight coefficients* (the *L1 norm* of the weights).
- *L2 regularization*—The cost added is proportional to the *square of the value of the weight coefficients* (the *L2 norm* of the weights). L2 regularization is also called *weight decay* in the context of neural networks. Don't let the different name confuse you: weight decay is mathematically the same as L2 regularization.

Regularization

Listing 4.6 Adding L2 weight regularization to the model

```
from keras import regularizers

model = models.Sequential()
model.add(layers.Dense(16, kernel_regularizer=regularizers.l2(0.001),
                        activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, kernel_regularizer=regularizers.l2(0.001),
                        activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

Regularization

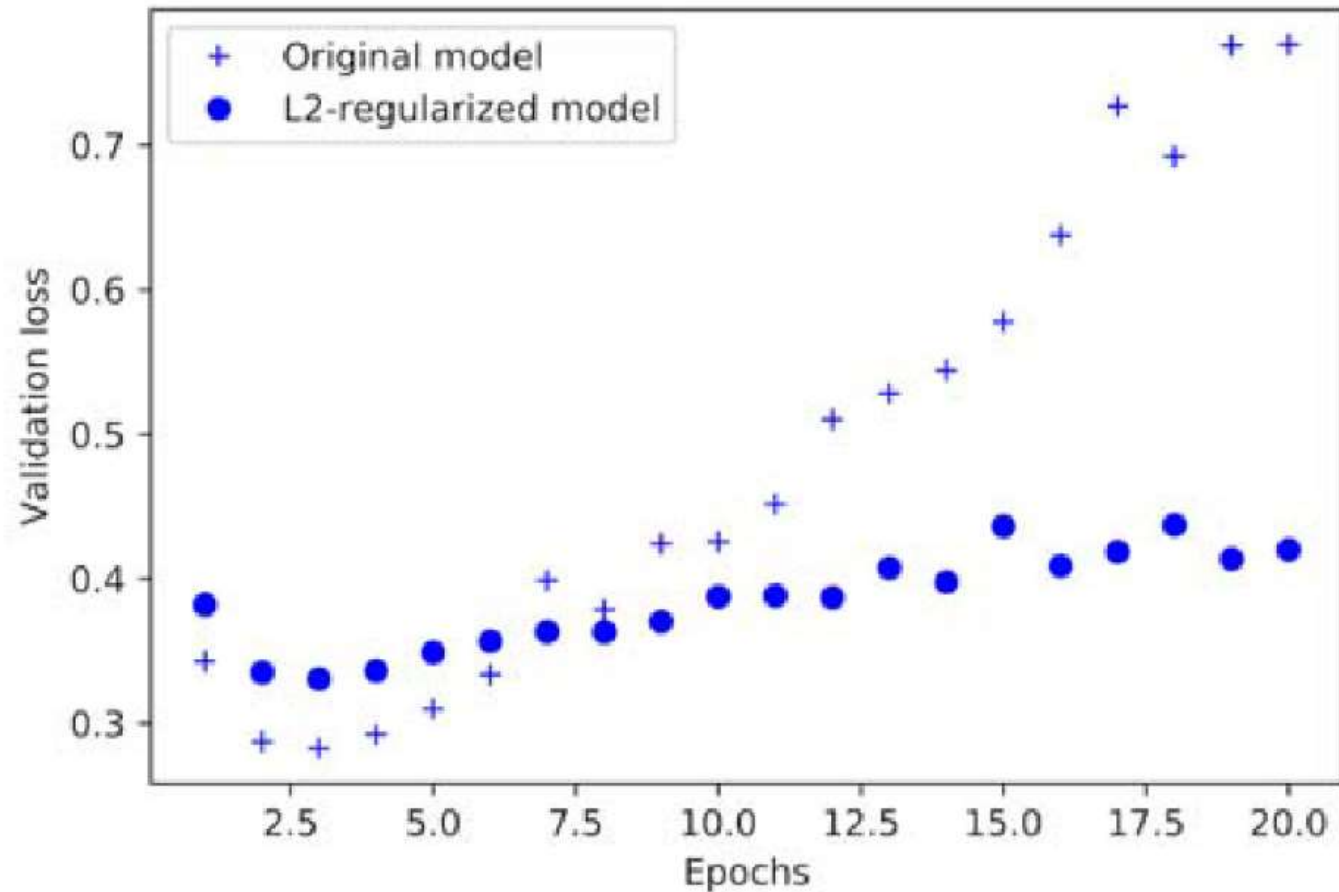
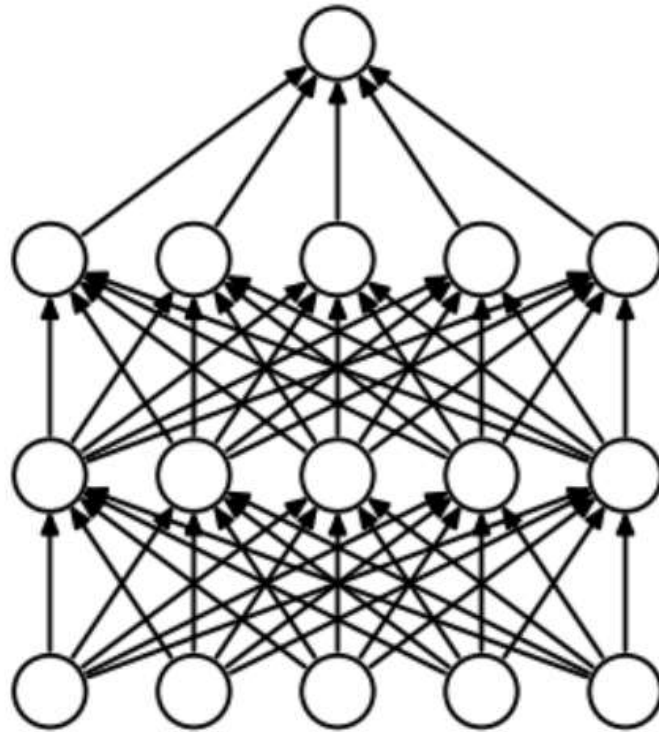


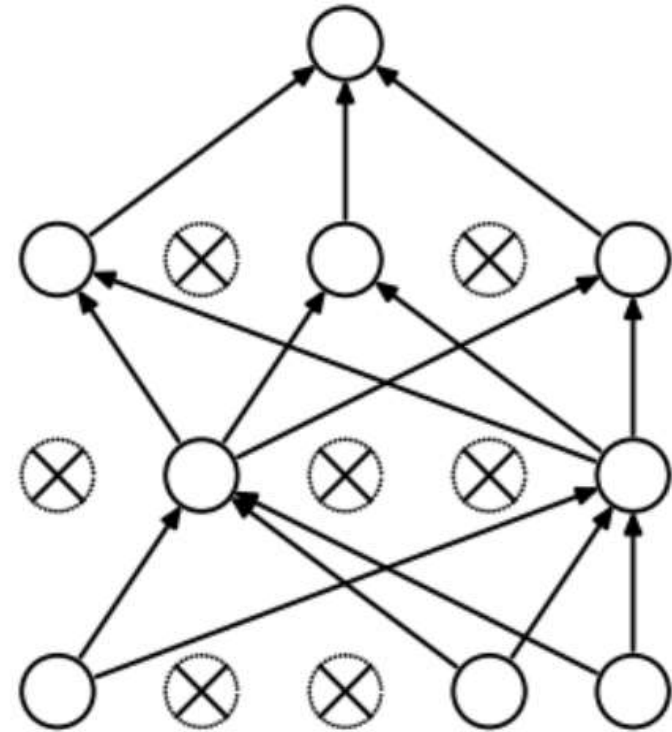
Figure 4.7 Effect of L2 weight regularization on validation loss

Regularization

- Adding dropout



(a) Standard Neural Net



(b) After applying dropout.

Regularization

- Adding dropout

Listing 4.8 Adding dropout to the IMDB network

```
model = models.Sequential()  
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))  
model.add(layers.Dropout(0.5))  
model.add(layers.Dense(16, activation='relu'))  
model.add(layers.Dropout(0.5))  
model.add(layers.Dense(1, activation='sigmoid'))
```


Regularization

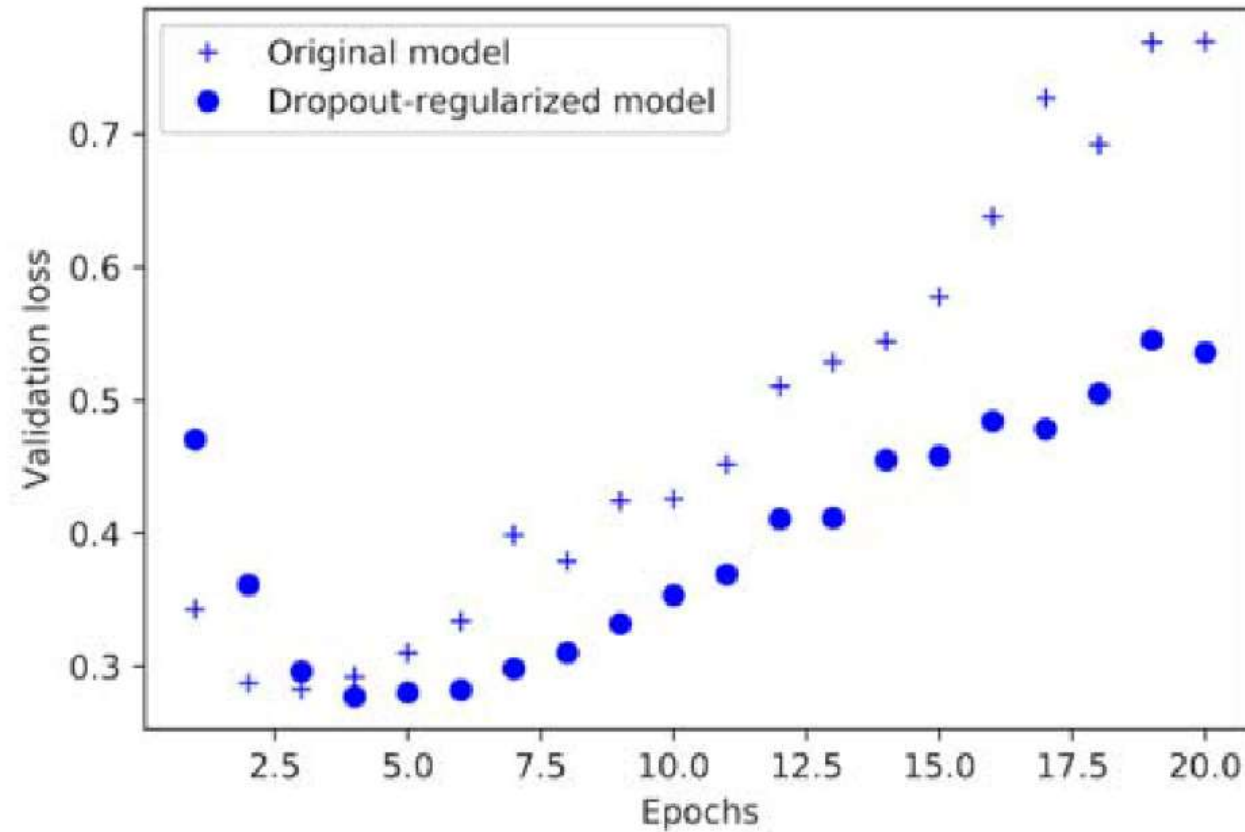


Figure 4.9 Effect of dropout on validation loss

To fight with overfitting, increase generalization and discourage optimization..

- Get more training data
- Reduce network capacity
- Add weight regularization
- Add dropout

Universal Workflow of Machine Learning Problems

- **Define problem and dataset**
 - Data inputs outputs
 - Binary classification, regression, multiclass?
 - Non-stationary data consideration

Universal Workflow of Machine Learning Problems

- **Choosing a measure of success**
 - Accuracy for class-balance problems
 - Precision and recall for class-imbalance problems

Universal Workflow of Machine Learning Problems

- **Deciding on an evaluation protocol**
 - Hold out cross validation: loads of data available
 - K-Fold cross validation: too few samples for hold out cross validation
 - Iterated K-Fold: for high accurate model too little data, say in 100s

Universal Workflow of Machine Learning Problems

- **Preparing your data**
 - Covert data in tensors
 - Range data to $[-1,1]$ or $[0,1]$
 - Normalize all features, bring to same range
 - Feature engineering if required

Universal Workflow of Machine Learning Problems

- **Developing a model that does better than baseline**
 - Random classification accuracy 0.5
- You hypothesize that your outputs can be predicted given your inputs.
- You hypothesize that the available data is sufficiently informative to learn the relationship between inputs and outputs.

Universal Workflow of Machine Learning Problems

- *Last-layer activation*—This establishes useful constraints on the network's output. For instance, the IMDB classification example used `sigmoid` in the last layer; the regression example didn't use any last-layer activation; and so on.
- *Loss function*—This should match the type of problem you're trying to solve. For instance, the IMDB example used `binary_crossentropy`, the regression example used `mse`, and so on.
- *Optimization configuration*—What optimizer will you use? What will its learning rate be? In most cases, it's safe to go with `rmsprop` and its default learning rate.

Universal Workflow of Machine Learning Problems

Table 4.1 Choosing the right last-layer activation and loss function for your model

Problem type	Last-layer activation	Loss function
Binary classification	<code>sigmoid</code>	<code>binary_crossentropy</code>
Multiclass, single-label classification	<code>softmax</code>	<code>categorical_crossentropy</code>
Multiclass, multilabel classification	<code>sigmoid</code>	<code>binary_crossentropy</code>
Regression to arbitrary values	None	<code>mse</code>
Regression to values between 0 and 1	<code>sigmoid</code>	<code>mse</code> or <code>binary_crossentropy</code>

Universal Workflow of Machine Learning Problems

- **Scaling up: developing a model that overfits**
 - Cross the border that exists between optimization and generalization
 - Border between overfitting and underfitting
- 1 Add layers.
 - 2 Make the layers bigger.
 - 3 Train for more epochs.

Universal Workflow of Machine Learning Problems

- **Regularizing your model and tuning hyper parameters**

- This step deserve your highest attention

- Add dropout.
- Try different architectures: add or remove layers.
- Add L1 and/or L2 regularization.
- Try different hyperparameters (such as the number of units per layer or the learning rate of the optimizer) to find the optimal configuration.
- Optionally, iterate on feature engineering: add new features, or remove features that don't seem to be informative.